Tomographic Neutron Absorption Models (TNAM): User Guide

Timothy Long, George Wood, Harry Lane, Sylvia Britto

August 2025

1 Introduction

This Julia program is built to correct neutron scattering data by taking into consideration neutron absorption and the varying flux incident on the sample at different angles. Both corrections are accomplished with a Monte Carlo approach. The absorption correction calculates the attenuation factor for each detector and each energy and divides the data by the corresponding attenuation factor. The flux correction calculates the area of the sample's projection onto the y-z plane (perpendicular to the incoming neutron beam) and divides all the data from a single .nxspe file by the corresponding area. As a result of this, the absorption correction is significantly more computationally-intensive. Spacial information about the sample, key to both corrections, is provided by exploiting neutron tomography to image the crystal(s).

2 Contents of TNAM Repository

Non-trivial contents of the TNAM repository include:

- **src** : Folder containing the key code the program uses.
  - **DataCorrection.jl** : This file is the main program and contains functions that will correct data for neutron flux and absorption individually and together.
  - **gen_crystal.jl** : Module containing functions for the attenuation calculation for a general sample morphology.
  - **nxspe.jl** : Module containing functions to extract and manipulate the data taken from the .nxspe files.
  - **projections.jl** : Module containing functions to aid in calculations of orthographic projection areas and rotating samples.
  - **sampling.jl** : Module containing functions that aid in determining random coordinates within the sample.
  - **sin_crystal.jl** : Module containing functions for the attenuation calculation for a single, convex sample morphology.
- **test** : Folder containing program tests and test data.
  - **runtests.jl** : Julia file containing tests checking the calculations of path lengths and attenuation factors.
  - **Test_STLs** : Folder containing dummy .stl files used for testing.
  - **Test_Notebooks** : Folder containing Julia notebooks of early tests of the various aspects of the program.
    - **AttenTest1.ipynb** : Julia notebook containing the calculation of attenuation factors using 1 MC sampling point and for 1 .nxspe file.
    - **GeneralAttenTest2.ipynb** : Julia notebook containing the calculation of attenuation factors using multiple MC sampling points and for 1 .nxspe file. Works for general sample morphology.
    - **MC_Sampling.ipynb** : Julia notebook containing the generation of coordinates inside the sample.
    - **OrthographicProjections.ipynb** : Julia notebook containing the calculation of the area of the sample's projection on the y-z plane.
    - **SingleAttenTest.ipynb** : Julia notebook containing the calculation of attenuation factors using multiple MC sampling points and for 1 .nxspe file. Works for single, convex sample morphology.

3 Accessing in IDAaaS

Due to the computationally-heavy nature of the program, it must be run on IDAaaS for reasonable running times. The user must ensure that the same version of Julia, currently 1.11.6, is being used for package compatibility reasons. This can be checked by typing `VERSION` into the Julia REPL. If a different version is being used on the IDAaaS workspace, download juliaup by running `curl -fsSL https://install.julialang.org | sh` in the terminal. Upon restarting the terminal, one should be able to choose the Julia version by writing `juliaup add 1.11.6` in the terminal. To load this up-to-date version of Julia, enter: `julia --threads=auto`. This step is required to allow for threading.

The following files must be uploaded from the repository into the folder TNAM for functionality.

- src
- Project.toml
- Manifest.toml

Finally, the user must enter the Julia REPL (inside the TNAM directory) and enter the Pkg REPL, via: `]`. To activate the Julia environment use: `activate .`. Staying in the Pkg REPL, the user must then ensure that all the correct packages are installed using: `instantiate`.

```
mpi/mpich-x86_64 has been loaded. To unload type the following into the terminal: module unload mpi/mpich-x86_64
[tl1163911@host-172-16-100-56 tl1163911]$ cd Desktop/TNAM
[tl1163911@host-172-16-100-56 TNAM]$ julia --threads=auto
               _
   _       _ _(_)_     |  Documentation: https://docs.julialang.org
  (_)     | (_) (_)    |
   _ _   _| |_  __ _   |  Type "?" for help, "]?" for Pkg help.
  | | | | | | |/ _` |  |
  | | |_| | | | (_| |  |  Version 1.9.3 (2023-08-24)
 _/ |\__'_|_|_|\__'_|  |  Official https://julialang.org/ release
|__/                   |

julia> Threads.nthreads()
64

(v1.9) pkg> activate .
  Activating project at `/mnt/ceph/home/tl1163911/Desktop/TNAM`

(TNAM) pkg> instantiate
```

Figure 1: Preparing the Julia environment in IDAaaS. Assumes the correct version of Julia is already in place.

Upon evaluating the file in the Julia REPL, via: `include("src/DataCorrection.jl")`, the functions that correct for flux (`correct_f()`), correct for absorption (`correct_a()`) and correct for both (`correct_af()`) are accessible.

4 User Inputs

The function that corrects each .nxspe file for neutron flux, `correct_f(nxspes, input_file_dir, file_start, file_end, output_file_dir, ψs, sample; n_flux, seed)`, has the following inputs.

- **nxspes** : Vector with integer elements containing the variable part of the names of each .nxspe file. For instance, for LET .nxspe files of the form `LET104215_3.7meV_1to1.nxspe`, `104215` is the variable aspect.
- **input_file_dir** : String describing the path to the folder containing the input .nxspe files.
- **file_start** : String describing the template of the start of each .nxspe file. For this LET example, this would be `"LET"`.
- **file_end** : String describing the template of the end of each .nxspe file. For the LET example, this would be `"_3.7meV_1to1.nxspe"`.

- **output_file_dir** : String describing the path to the folder which will contain the output .nxspe files.
- ψ**s** : Vector with real elements containing the sample rotation angle, ψ, corresponding to each .nxspe file, in degrees.
- **sample** : String describing the path to the sample's .stl file. Should have units of mm, represent the sample at 0 degrees and have an origin at the centre of rotation. These conditions may be satisfied via manipulation in Avizo.
- **n_flux** : Optional integer equal to the number of MC sample point used in the flux correction. Default value of 100,000.
- **seed** : Optional integer random seed.

The function that corrects the data for absorption, `correct_a(nxspes, input_file_dir, file_start, file_end, output_file_dir, ψs, sample, complex, μ_ref; en_ref, n_abs, seed)`, has the following extra inputs.

- **complex** : Bool describing the functions that the program will use. `true` will perform the correction for a general sample morphology. The program for the general crystal(s) calculates every intersection point, allowing for neutron paths to exit and re-enter the sample. `false` will perform the correction for a single, convex sample morphology. The program for the single crystal assumes only one intersection point and is, therefore, considerably faster.
- μ**_ref** : Real number equal to the attenuation coefficient at the reference energy, in cm^{-1}.
- **en_ref** : Optional real number equal to the reference energy, in meV. Default of 25.3 meV.
- **n_abs** : Optional integer equal to the number of MC sample point used in the absorption correction. Default value of 1,000.

The function that corrects the data for both neutron flux and absorption, `correct_af(nxspes, input_file_dir, file_start, file_end, output_file_dir, ψs, sample, complex, μ_ref; en_ref, n_abs, n_flux, seed)`, uses all the arguments listed above.

5 Program Outputs

The program outputs .nxspe files into a folder chosen by the user. These files are copies of the original .nxspe files except with the corrected data and rotation angle, ψ, added in place of the previous data. These new files have the same names but with `aC_`, `fC_` or `afC_` in front depending on whether they have been absorption-, flux-, or absorption and flux-corrected.

6 Error Outputs

Errors may be outputted in this program if the data outputted is likely to be inaccurate. Excluding trivial errors due to the specific types of inputs required, the following error messages may be seen.

- `AssertionError: The path length could not be calculated at any sample point (for a specific final wavevector). Try increasing n_abs.` For each detector and energy, the attenuation factor is calculated using many MC sample points. This error means that the attenuation factor could not be calculated (for at least one detector and energy combination) since the post-scattering neutron path length could not be defined for any sample point. This may be down to the Moller-Trumbore algorithm believing that no face has been intersected, or that an even number of intersections has occurred (usually signifying the coordinate is outside the crystal(s)). The incorrect calculation that no faces were intersected is likely due to floating point precision errors. An even number of intersections may be wrongly calculated due to the program's duplicate checker. In some cases, it was found that path lengths could be calculated twice due to neutron paths near a vertex and multiple faces claiming that the neutron intersected with their face. Thus, a duplicate path length remover was required. Currently, this code checks if path lengths are within 0.0001 of one another and

removing one if so. However, these path length differences are dependent on the magnitude of the sample coordinates. For instance, an .stl file with coordinates of the order of 0.000001 are all going to be within 0.0001 of one another. Therefore, if a significant number of path lengths are being outputted as `nothing` it may be worth adjusting this value to better suit your sample. It may also be worth checking that the MC coordinates are, indeed, within the crystal(s). Of course, increasing n_abs increases the number of coordinates and means a path length is more likely to be defined for at least one of them.

- `AssertionError: The proportion of n_abs (MC sample points used in the absorption correction) being used is *av_acc_rate*. If this is not enough accuracy, please increase n_abs.` This error is triggered when less than 80% of the sample points can be used. A sample point is not used when the post-scattering neutron path length, L_f, from it is not defined (ie outputs `nothing`). The corrected .nxspe files are still outputted. This error serves as a warning in case the outputted files are not accurate enough given the acceptance rate of sample points.

7 Typical Run Times and Scaling

The following run times were collected using the LET: Excitations Single Crystal Large workspace on IDAaaS with 64 threads and 247GB RAM. LET has 98,304 detectors with 320 energy bins but only around 1% of the data was not zero or NaN. Furthermore, the .stl file describing the sample had around 2,200 faces. Finally, 100 LET files were corrected. With this configuration, it took around 1 hour (in single crystal mode) and 4 hours (in general crystal mode) to correct the data using n_abs = 1,000 and n_flux = 10,000. In comparison, correcting just neutron flux, with n_flux = 100,000, took around 10 seconds.

The total run time depends strongly on the following quantities.

- **n_faces** : The number of faces comprising the mesh describing the sample surface. This can be chosen in Avizo when creating the .stl file. The run time should scale linearly with the number of faces.
- **n_abs** : The number of MC sample points at which the attenuation factor is calculated for the absorption correction. The run time should roughly scale linearly with n_abs if the absorption correction is being performed.
- **n_flux** : The number of MC sample points used in the flux correction. Assuming the absorption correction is also being done, the run time should be approximately independent of n_flux as it is a much faster process. If not, the run time should scale linearly with n_flux.
- **n_files** : The number of .nxspe files being corrected. The variation of the run time with the number of files depends on the number of threads being used. Each thread is assigned to a different file. If n_files < n_threads, the number of files can be increased without drastically affecting the run time. As soon as n_files exceeds n_threads, the run time should increase.
- **complex** : Complexity of the sample (single or general program needed). The program for a general crystal morphology appears to be approximately a factor of 3-4x slower. This dependence only arises if the data is being corrected for absorption.
- **File Sparsity** : Number of zero and NaN elements of the data in the .nxspe file. The running time should be inversely proportional to the file sparsity for the absorption correction. The file sparsity should have little effect on the flux correction.